

Cleft - เอกสารสรุปโปรเจกต์ฉบับ Portfolio

อัปเดตล่าสุด: 5 กันยายน 2026

สถานะ: deploy ใช้งานจริงแล้วที่ <https://cleft-usb2.vercel.app> โดย Frontend อยู่บน Vercel Hobby, Backend และ PostgreSQL อยู่บน Render Free; โค้ดบน main ผ่าน GitHub Actions ทั้ง Backend, Frontend, browser tests และ design prototype

จุดประสงค์: ใช้ทำความเข้าใจระบบ เตรียมอธิบายผลงาน ประเมินขอบเขตการ scale และเป็น checklist ด้านความปลอดภัย/ค่าใช้จ่ายก่อนนำเสนอ portfolio

1. Executive Summary

Cleft คือ Full Stack web application สำหรับแบ่งค่าใช้จ่ายของกลุ่ม ผู้ใช้สร้างบิล เพิ่มรายการและจำนวน เพิ่มสมาชิก กำหนดว่าสมาชิกแต่ละคนร่วมจ่ายทุกรายการหรือเฉพาะรายการ ตรวจสอบยอดที่คำนวณแบบรายสัปดาห์ แลกสลิป และยืนยันบิลลง PostgreSQL เพื่อดูประวัติภายหลัง

โปรเจกต์นี้แสดงความสามารถที่เหมาะสมกับ portfolio ด้านต่อไปนี้:

- ออกแบบระบบแยก Frontend, Backend และ Database อย่างชัดเจน
- สร้าง authentication ด้วย Google OAuth, JWT และ HTTP-only cookies
- ออกแบบ business rule สำหรับการหารเงินที่ตรวจสอบผลรวมได้
- จัดการข้อมูลสัมพันธ์ด้วย Prisma migration และ transaction
- ทำ responsive UI และมีหน้า demo ที่ไม่ต้องล็อกอิน
- ทำ automated quality gates ตั้งแต่ lint ถึง browser test
- เตรียม Docker Compose และ environment validation สำหรับการนำระบบไปรัน
- deploy แยก Frontend/Backend จริง พร้อม same-origin API proxy, Google OAuth และ production health checks

สิ่งที่ Cleft ไม่ได้อ้างว่าเป็นในสถานะปัจจุบัน:

- ไม่ใช่ระบบโอนเงินจริง
- ยังไม่ใช่ระบบ settlement หรือระบบติดตามหนี้ครบวงจร
- production ปัจจุบันจำกัดการ login ไว้ที่บัญชีเจ้าของเพียงบัญชีเดียว จึงเป็น private portfolio demo ไม่ใช่ระบบ multi-user สาธารณะ
- design prototype ใน `edit_front_end/` ไม่ใช่ production frontend

2. Problem และแนวคิดของผลิตภัณฑ์

การหารบิลด้วยมือมักผิดพลาดเมื่อสมาชิกแต่ละคนกินไม่เหมือนกัน มีหลายรายการ หรือราคาอาหารไม่ลงตัว Cleft แก้ปัญหานี้ด้วยการแยกข้อมูลเป็น item และ participant แล้วคำนวณผู้รับผิดชอบของแต่ละ item โดยตรง

เส้นทางหลักของผู้ใช้:

1. Login ด้วย Google
2. เพิ่มชื่อปาร์ตี้ วันที่ รายการ ราคา จำนวน และหมายเหตุ
3. เพิ่มสมาชิกและเลือก ALL หรือ PARTIAL
4. ตรวจสอบรายการที่ยังไม่มีผู้รับผิดชอบ
5. ดูยอดต่อคนและยอดรวมใน Summary
6. แนวนรูปสลิปถ้าต้องการ
7. Confirm เพื่อบันทึกข้อมูลแบบ transaction
8. ดูบิลล่าสุดจากหน้า Home

สำหรับ reviewer ที่ไม่ต้องการตั้งค่า OAuth มีหน้า /demo แสดงตัวอย่างบิลและผลการแบ่งโดยไม่อ่านหรือเขียนฐานข้อมูล

3. ขอบเขตความสามารถปัจจุบัน

3.1 Authentication และผู้ใช้

- เริ่ม Google OAuth ผ่าน /api/auth/google
- รับ callback ผ่านโดเมน Frontend ที่ /api/auth/google/callback
- ตรวจสอบอีเมลจาก Google แบบ exact match โดยไม่สนตัวพิมพ์ใหญ่เล็กกับ ALLOWED_LOGIN_EMAIL ก่อน upsert ผู้ใช้ ปัจจุบันจึงอนุญาตเฉพาะบัญชีเจ้าของ
- ใช้ access token และ refresh token ใน HTTP-only cookies
- หมุน refresh token เมื่อเรียก refresh endpoint
- เก็บเฉพาะ SHA-256 hash ของ refresh token ในฐานข้อมูล
- Logout พร้อม revoke refresh token ที่บันทึกไว้
- อ่านและแก้ไข username, avatar และ currency symbol

3.2 การสร้างและแบ่งบิล

- เพิ่ม แก้ไข และลบ item
- รองรับราคาต่อชิ้น quantity และ note
- ป้องกันชื่อ item ซ้ำโดยไม่สนตัวพิมพ์ใหญ่เล็ก
- เพิ่ม แก้ไข และลบ member
- ป้องกันชื่อ member ซ้ำ
- เปลี่ยนชื่อหรือลบ item พร้อมปรับ reference ของสมาชิก PARTIAL
- แบ่งแบบ ALL และ PARTIAL
- ป้องกัน Confirm ถ้ายังมี item ที่ไม่มีสมาชิกคนใดรับผิดชอบ
- กระจายเศษสตางค์แบบ deterministic เพื่อให้ยอดสมาชิกรวมเท่ากับยอดบิล
- แนวนรูป JPG, JPEG, PNG, GIF หรือ WebP ขนาดไม่เกิน 5 MB
- บันทึก Party, Item, Participant, Consumption และ History ภายใน transaction เดียว

3.3 ประสบการณ์ใช้งาน

- Draft ถูกเก็บใน `localStorage` ภายใต้ key `cleft-session`
- หน้า Home แสดงบิลปัจจุบัน ประวัติล่าสุด และกราฟการกระจายยอด
- หน้า Summary แสดงยอดรายคน stacked bar และ digital slip
- หน้า `/demo` ใช้ข้อมูลคงที่สำหรับนำเสนอ portfolio
- UI รองรับ desktop และ mobile
- มี loading state, toast และ auth guard

4. Technology Stack

ชั้นระบบ	เทคโนโลยี	หน้าที่หลัก
Production Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS	Routes, UI, local draft และ API client
Client Data	TanStack Query, Axios	Request cache, profile/history query และ refresh flow
Visualization	Recharts	Pie chart และสัดส่วนค่าใช้จ่าย
Production Backend	NestJS 12, TypeScript	REST API, validation, authentication และ business logic
Authentication	Passport, Google OAuth, JWT	Login, session, refresh และ logout
Database Access	Prisma 5	Schema, migration, query และ transaction
Database	PostgreSQL 16 ใน local/CI; Render PostgreSQL 18 ใน production	จัดเก็บผู้ใช้ บิล สมาชิก รายการ และประวัติ โดย migration เดียวกัน
Upload	Multer	จำกัดขนาด ตรวจสอบชนิด และเขียนไฟล์สลิป
Test	Vitest, Testing Library, Playwright	Unit, component, browser และ integration tests
Delivery	Docker, Docker Compose, GitHub Actions	Local stack และ automated CI
Design Archive	Vite, React	เก็บ UI prototype เพื่อแสดงพัฒนาการของงานออกแบบ

5. Repository Map

```
Project_github/
|-- frontend/           Production Next.js application
|   |-- app/            Home, Items, Members, Summary, Demo
|   |-- components/     Navigation, charts และ UI primitives
|   |-- hooks/          Party draft state และ localStorage
|   |-- lib/            API client, types และ bill calculation
|   |-- e2e/            Playwright browser tests
|   |-- test/           Vitest component/helper tests
|   |-- public/         Cleft icons และ PWA assets
|-- backend/           Production NestJS API
|   |-- src/auth/       OAuth, JWT, refresh และ logout
|   |-- src/users/      Profile API
|   |-- src/party/      Calculate, confirm และ history
|   |-- src/prisma/     Database service
|   |-- src/config/     Environment validation
|   |-- prisma/         Schema และ committed migrations
|   |-- scripts/        API integration smoke test
|-- edit_front_end/    Archived design prototype
|-- uploads/           Local/Docker slip storage
|-- scripts/           Project utilities
|-- .github/workflows/ci.yml Automated quality pipeline
|-- docker-compose.yml Full local stack
|-- PORTFOLIO.md       ทางลัดสำหรับ reviewer
|-- SUMMARY.html       เอกสาร HTML local (ถูก ignore จาก Git)
|-- output/pdf/SUMMARY.pdf PDF ที่ track บน GitHub สำหรับส่งประกอบ portfolio
```

ไฟล์ `.env` จริง, OAuth client JSON, build output, test report และไฟล์ upload ต้องไม่อยู่ใน Git ปัจจุบันตรวจทั้ง working tree และประวัติ commit แล้วไม่พบ Google Client Secret หรือไฟล์ `.env` จริงถูก track ส่วน `SUMMARY.html` เป็นไฟล์ local ที่ ignore ไว้ และส่งฉบับเดียวกันขึ้น GitHub ในรูป `output/pdf/SUMMARY.pdf`

6. System Architecture

```
Browser
|
| pages และ same-origin /api requests
v
Next.js Frontend :3000
|-- React UI
|-- TanStack Query และ Axios
|-- localStorage draft
|-- /demo แบบไม่ใช้บัญชี
|
| rewrite /api/:path*
v
NestJS Backend :4000
|-- Helmet, CORS, cookie parser
|-- Passport JWT guard
|-- DTO validation
|-- Auth, User และ Party modules
|-- Multer slip upload
v
Prisma ORM
|-- committed migration
|-- transaction ตอน Confirm
v
PostgreSQL :5432
```

Frontend ใช้ path `/api` เสมอ Next.js rewrite จะส่ง request ต่อไปยัง Backend ตาม `API_PROXY_TARGET` วิธีนี้ทำให้ client ไม่ต้องรู้ URL ภายในของ Backend และช่วยให้ cookies ถูกใช้งานใน origin เดียวกับหน้าเว็บ

7. Data Flow สำคัญ

7.1 Google Login

```
User clicks Google login
-> GET /api/auth/google
-> Next.js rewrite
-> NestJS Google guard
-> Google consent
-> /api/auth/google/callback ผ่าน Frontend domain
-> compare Google email กับ ALLOWED_LOGIN_EMAIL
-> reject ก่อนแตะฐานข้อมูลหากไม่ตรง
-> User upsert ใน PostgreSQL
-> access/refresh tokens
-> HTTP-only cookies
-> redirect กลับ Frontend
```

Callback ผ่าน Frontend URL เพื่อให้ response ที่ตั้ง cookie กลับมาบน origin ที่ผู้ใช้เปิดอยู่ การตั้งค่า `GOOGLE_CALLBACK_URL` ใน Google Cloud ต้องตรงกับค่าจริงทุกตัวอักษร และ OAuth Client ต้องยังไม่ถูกลบ ปัจจุบันระบบ fail closed: ถ้าไม่มี `ALLOWED_LOGIN_EMAIL` หรืออีเมลไม่ตรง ระบบจะตอบ Unauthorized และไม่สร้าง user record

7.2 Draft Bill

Items/Members forms

- > React state
- > usePartySession
- > localStorage: cleft-session
- > bill-display helpers
- > Summary preview

Draft ยังไม่ถูกส่งเข้า Database จนกว่าจะ Confirm จึงแก้ไขได้เร็วและไม่สร้างข้อมูลค้างใน server แต่ draft จะอยู่เฉพาะ browser และอุปกรณ์นั้น

7.3 Confirm Bill

Summary validation

- > normalize quantity เป็น line total
- > multipart FormData: JSON payload + optional slip
- > JWT guard
- > JSON parse และ nested DTO validation
- > business-rule validation
- > Prisma transaction
- > Party + Items + Participants + Consumptions + History
- > clear local draft

ถ้าการเขียนฐานข้อมูลส่วนใดล้มเหลว Prisma transaction จะ rollback record ในฐานข้อมูลทั้งหมด

8. Bill Calculation และ Data Integrity

8.1 Split Rules

- `ALL` : สมาชิกคนนั้นถูกเพิ่มเป็น assignee ของทุก item
- `PARTIAL` : สมาชิกคนนั้นเป็น assignee เฉพาะชื่อ item ที่อยู่ใน `itemNames`
- `PARTIAL` ที่ไม่เลือก item มียอด 0 แต่ระบบจะไม่อนุญาตให้มี item ใดไม่มี assignee เลย

8.2 วิธีคำนวณ

Frontend แปลงราคาต่อชิ้นเป็น line total ก่อนส่ง Backend:

```
lineTotal = unitPrice x quantity
```

สำหรับแต่ละ item ระบบทำงานเป็นจำนวนเต็มหน่วยสตางค์:

```
itemCents = round(lineTotal x 100)
baseShare = floor(itemCents / assigneeCount)
remainder = itemCents mod assigneeCount
```

ทุกคนได้รับ `baseShare` และ `remainder` ถูกแจกทีละ 1 สตางค์ตามลำดับ `assignee` ที่คงที่ ผลรวมยอดรายคนจึงเท่ากับ ยอดบิลเสมอ

ตัวอย่าง 100 บาท หหาร 3 คน:

คน	ยอด
Alice	33.34
Bob	33.33
Cara	33.33
รวม	100.00

8.3 Validation ที่ป้องกันข้อมูลผิด

- Party ต้องมีชื่อและวันที่ที่ parse ได้
- ต้องมีอย่างน้อย 1 item และ 1 participant
- ราคา item ต้องเป็นจำนวนบวก
- ชื่อ item และ participant ต้องไม่ว่างและไม่ซ้ำหลัง trim/case normalization
- `PARTIAL.itemNames` ต้องอ้างถึง item ที่มีอยู่จริง
- ทุก item ต้องมี assignee อย่างน้อยหนึ่งคน
- Backend คำนวณและ validate เอง ไม่เชื่อผลที่คำนวณจาก browser

9. REST API

Method	Endpoint	Auth	หน้าที่
GET	<code>/health</code>	ไม่ต้อง	Backend health check
GET	<code>/auth/google</code>	ไม่ต้อง	เริ่ม Google OAuth
GET	<code>/auth/google/callback</code>	Google guard	รับ OAuth result และตั้ง cookies
GET	<code>/auth/session</code>	JWT	อ่าน session ปัจจุบัน
POST	<code>/auth/refresh</code>	Refresh cookie	ตรวจและหมุน refresh token
POST	<code>/auth/logout</code>	JWT	Revoke token และล้าง cookies

Method	Endpoint	Auth	หน้าที่
GET	/users/profile	JWT	อ่าน profile
PATCH	/users/profile	JWT	แก้ profile
POST	/party/calculate	JWT	Validate และคำนวณบิล
POST	/party/confirm	JWT	บันทึกบิลและ optional slip
GET	/party/history	JWT	ประวัติล่าสุด 3 รายการ
GET	/party/history?all=true	JWT	ประวัติทั้งหมด

เมื่อ request ได้ 401 Axios interceptor จะพยายาม refresh หนึ่งครั้ง และ request ที่เกิดพร้อมกันจะรอ refresh promise เดียวกัน เพื่อลดการส่ง refresh ซ้ำ

10. Database Model

```

User 1 -----< History >----- 1 Party
      |
      +-----< Item
      |
      +-----< Participant
                  |
            Item >-----< Consumption
  
```

User

เก็บ Google identity, email, username, avatar, currency symbol และ refresh token hash โดย googleId และ email เป็น unique

Party

เก็บชื่อ วันที่ ยอดรวม URL ของสลิป และเวลา created/updated

Item

เก็บชื่อ line total และ note หลัง Confirm ปัจจุบัน quantity ถูกแปลงรวมใน price และบันทึกเป็นข้อความใน note

Participant

เก็บชื่อและ split type ของสมาชิกใน Party

Consumption

เป็น many-to-many join ระหว่าง Participant และ Item ใช้เก็บ item selection ของสมาชิก PARTIAL ด้วย composite primary key

History

เชื่อม User กับ Party ที่ผู้ใช้นั้นสร้าง ใช้เป็น ownership path สำหรับหน้า history

Relations หลักใช้ onDelete: Cascade เพื่อลบ record ลูกเมื่อ parent ถูกลบ

11. Authentication และ Security Posture

มาตรการที่มีแล้ว:

- HTTP-only cookies ลดการอ่าน token จาก JavaScript
- Secure cookie ใน production
- SameSite ตั้งค่าได้เป็น lax หรือ none
- Refresh token ถูก hash ก่อนเก็บ Database
- Refresh token rotation และ revoke ตอน logout
- Helmet security headers
- CORS จำกัดตาม FRONTEND_URL
- Global ValidationPipe ใช้ whitelist และ transform
- Confirm DTO ใช้ forbidNonWhitelisted
- JWT guard ครอบ profile และ party endpoints
- จำกัด upload 5 MB และตรวจ MIME prefix กับ extension
- Production validation บังคับ JWT secret ยาวอย่างน้อย 32 ตัวอักษร
- Secrets จริงไม่อยู่ใน Git

สิ่งที่ต้อง harden ก่อนเปิด public production:

1. เพิ่ม rate limiting ให้ auth, refresh, calculate, confirm และ upload
2. ประเมิน CSRF protection เพิ่มเติม โดยเฉพาะ deployment ที่ต้องใช้ SameSite=None
3. ตรวจ file signature หรือ magic bytes แทนการเชื่อ MIME/extension อย่างเดียว
4. ย้าย slip ไป private object storage และใช้ signed URL เมื่อเหมาะสม
5. ลบไฟล์ที่ Multer เขียนแล้ว หาก payload validation หรือ transaction ล้มเหลว
6. เพิ่ม log, error tracking และ security monitoring ที่ไม่บันทึก token/secret
7. วางนโยบาย backup, restore และ credential rotation

12. Automated Testing และ CI

GitHub Actions ทำงานทุก push ไป main และทุก pull request แบ่งเป็น 3 jobs:

12.1 Backend quality and integration

- `npm ci`
- Prisma Client generation
- ESLint
- TypeScript typecheck
- Vitest unit tests
- NestJS production build
- PostgreSQL 16 service
- `prisma migrate deploy`
- API integration smoke test

Backend unit tests ปัจจุบัน 7 เคส แบ่งเป็น PartyService 4 เคสและ AuthService 3 เคส ครอบคลุม:

- การแบ่ง `ALL` และ `PARTIAL`
- การแจกเศษสตางค์แบบ deterministic
- การปฏิเสธ item ที่ไม่มี assignee
- การปฏิเสธข้อผิดพลาด normalization
- อนุญาตอีเมลที่กำหนดโดยไม่สนตัวพิมพ์ใหญ่เล็ก
- ปฏิเสธ Google account อื่นก่อนเรียก Prisma upsert
- ปฏิเสธทุกบัญชีเมื่อขาด `ALLOWED_LOGIN_EMAIL`

Integration smoke test ครอบคลุม health, authenticated session, profile update, calculation edge cases, confirm, history และ cleanup ข้อมูลทดสอบ

12.2 Frontend quality and browser tests

- `npm ci`
- ESLint
- TypeScript typecheck
- Vitest unit/component tests
- Next.js production build
- Playwright Chromium installation
- Playwright E2E

Frontend unit/component tests ปัจจุบัน 5 เคส ครอบคลุม calculation/display helpers และ shared Button component

Browser tests ปัจจุบัน 3 เคส:

- Signed-out login page แสดง action ที่ถูกต้อง
- Reviewer เปิด `/demo` ได้โดยไม่ต้อง authenticate
- Protected route redirect ผู้ใช้ที่ยังไม่ login

12.3 Archived design prototype

- `npm ci`
- Vite production build
- Vendor code splitting เพื่อลด bundle หลัก

สถานะ CI ล่าสุด ณ วันที่เอกสารนี้ถูกสร้าง: ผ่านทั้ง 3 jobs และ Playwright ผ่าน 3 tests

13. Local และ Docker Runbook

Docker Compose

1. สร้าง `backend/.env` จากตัวอย่างและใส่ค่าจริงสำหรับ local
2. ตรวจ Google callback ให้เป็น `http://localhost:3000/api/auth/google/callback`
3. รัน:

```
docker compose up -d --build
docker compose ps
```

ตรวจระบบ:

```
Frontend: http://localhost:3000
Demo:     http://localhost:3000/demo
Health:   http://localhost:3000/api/health
Backend:  http://localhost:4000/health
```

ลำดับ startup:

```
PostgreSQL healthy
-> Backend runs prisma migrate deploy
-> Backend healthcheck passes
-> Frontend starts
```

Native quality checks

```
cd backend
npm ci
npm run prisma:generate
npm run lint
npm run typecheck
npm run test:unit
npm run build

cd ../frontend
npm ci
npm run lint
npm run typecheck
npm run test:unit
npm run build
npm run test:e2e

cd ../edit_front_end
npm ci
npm run build
```

14. Environment Variables

Backend

Variable	ความหมาย
DATABASE_URL	PostgreSQL connection string
PORT	Backend port ค่าเริ่มต้น 4000
FRONTEND_URL	Allowed origin และ redirect หลัง login
GOOGLE_CLIENT_ID	Google OAuth client ID
GOOGLE_CLIENT_SECRET	Google OAuth client secret
GOOGLE_CALLBACK_URL	Callback ที่ลงทะเบียนใน Google Cloud
ALLOWED_LOGIN_EMAIL	อีเมล Google เพียงบัญชีเดียวที่อนุญาตให้สร้าง session; comparison แบบ trim และ case-insensitive และเป็น required variable เพื่อให้ระบบ fail closed
JWT_ACCESS_SECRET	Signing secret ของ access token
JWT_REFRESH_SECRET	Signing secret ของ refresh token
JWT_ACCESS_TTL	อายุ access token ค่าเริ่มต้น 15m

Variable	ความหมาย
JWT_REFRESH_TTL	อายุ refresh token ค่าเริ่มต้น 7d
COOKIE_SAME_SITE	lax หรือ none

Frontend

Variable	ความหมาย
API_PROXY_TARGET	Backend URL สำหรับ Next.js rewrite

API_PROXY_TARGET ถูกอ่านตอน Frontend build หากเปลี่ยน Backend URL ต้อง rebuild Frontend image

15. จุดที่ผิดพลาดและวิธีวิเคราะห์

ตารางนี้เรียงจากอาการที่เจอบ่อยในการพัฒนาและ deployment จริงของโปรเจกต์ ไม่ใช่เพียงข้อสันนิษฐานจาก framework

อาการ	สาเหตุที่ควรตรวจ	แนวทางแก้/ป้องกัน
Backend build ผ่านแต่ start ไม่ได้	ตัวแปร required ขาด โดยเฉพาะ ALLOWED_LOGIN_EMAIL , database URL หรือ JWT secret; validation ทำงาน ตอน bootstrap	เทียบ environment กับ backend/.env.example และ CI workflow ทุกครั้งที่เพิ่มตัวแปรใหม่ อย่าตั้ง fallback ที่ทำให้ auth เปิดกว้างโดยไม่ตั้งใจ
GitHub Actions ล้มเหลวหลัง เพิ่ม env validation	production config ถูก อัปเดตแล้ว แต่ test job ไม่มีค่าจำลอง	เพิ่มค่าที่ไม่ใช่ secret ใน job-level env ; ใช้อีเมล ตัวอย่างใน CI ไม่ใช่อีเมลจริงใน workflow สาธารณะ
Docker ทา backend/.env ไม่พบ	Compose อ้าง env_file ที่ถูก ignore และไม่มีในเครื่องใหม่	คัดลอกจาก backend/.env.example แล้วเติมค่า จริง ห้าม commit .env
Database ต่อได้แต่ไม่มีตาราง	ยังไม่ได้ apply migration หรือใช้ database คนละตัว	ตรวจ host/schema แล้วใช้ committed migration กับ prisma migrate deploy ; ไม่ใช่ prisma db push เป็นขั้น production
Frontend container ติดต่อ localhost:4000 ไม่ได้	localhost ภายใน container หมายถึง container ตัวเอง	ใน Compose ใช้ service DNS http://backend:4000 ; บน Vercel ใช้ URL ของ Render

อาการ	สาเหตุที่ควรตรวจ	แนวทางแก้/ป้องกัน
เปลี่ยน Backend URL แล้ว rewrite ยังไปที่เดิม	API_PROXY_TARGET ถูกอ่านตอน Next.js build	ตั้ง environment ให้ถูก environment แล้ว rebuild/redeploy Frontend ไม่ใช่ restart อย่างเดียว
Google แสดง deleted_client	OAuth Client ID ใน runtime ถูกลบหรือยังชี้ credential เก่า	สร้าง Web OAuth Client ใหม่ ใส่ redirect URI ที่ตรงอัปเดตทั้ง ID/Secret บน Render และสร้าง deployment ใหม่ จากเหตุการณ์จริง restart อย่างเดียวอาจยังใช้ environment snapshot เก่า
Google แสดง redirect_uri_mismatch	URI ต่างกันแม้เพียง scheme, host, path หรือ slash ท้าย	ค่าปัจจุบันต้องตรงกับ https://cleft-usb2.vercel.app/api/auth/google/callback ทุกตัวอักษร
Google แสดง access denied หรือผู้ใช้ทดสอบเข้าไม่ได้	แอปอยู่สถานะ Testing แต่บัญชีไม่อยู่ใน Test users หรือ authorization ทดสอบหมดอายุ	เพิ่มบัญชีใน Google Auth Platform Audience; โหมด Testing รองรับสูงสุด 100 test users และ authorization ของ test user มีอายุ 7 วัน
บัญชี Google อื่นถูกตอบ Unauthorized	เป็นพฤติกรรมตั้งใจจาก email allowlist ไม่ใช่ OAuth เสีย	สำหรับ portfolio ปัจจุบันให้ใช้บัญชีเจ้าของเท่านั้น หากเปิดสาธารณะต้องเปลี่ยนนโยบาย allowlist และเพิ่ม authorization model ก่อน
OAuth ผ่านแต่หน้า Frontend ยังเหมือนไม่ login	cookie ไม่ถูกตั้ง/ส่ง หรือ callback ไปคนละ origin	ตรวจ FRONTEND_URL , GOOGLE_CALLBACK_URL , HTTPS, Secure, SameSite และ response Set-Cookie ; สถาปัตยกรรมนี้ตั้งใจให้ callback ผ่าน Frontend origin
Request แรกช้ามากแต่ครั้งต่อไปเร็ว	Render Free spin down หลังไม่มี traffic 15 นาที	อธิบาย reviewer ว่าเป็น cold start ของแผนฟรี ไม่ใช่ frontend ค้าง; หลีกเลี่ยง ping ปลุกตลอดเวลาที่อาจกิน free instance hours
ข้อมูลหายหรือ database เข้าไม่ได้หลังใช้งานประมาณหนึ่งเดือน	Render Free Postgres หมดอายุ 30 วัน	Export ข้อมูลก่อน 4 ตุลาคม 2026, สร้างฐานฟรีใหม่แล้ว migrate/restore หรือย้ายไปฐานข้อมูลถาวรก่อนวันหมดอายุ
รูปสลิปหายหลัง deploy/restart/spin down	Render Free filesystem เป็น ephemeral	อย่าใช้สลิปเป็นหลักฐานถาวรในแผนปัจจุบัน; ก่อนเปิดจริงให้ย้ายไป object storage และเก็บ object key ใน database
มีไฟล์ upload ค้างแต่ไม่มี Party	Multer เขียนไฟล์ก่อน controller parse/validate payload และ transaction ไม่ครอบ filesystem	ตรวจ payload ก่อน upload เมื่อออกแบบใหม่ หรือ cleanup ไฟล์เมื่อ validation/transaction ล้มเหลว

อาการ	สาเหตุที่ควรตรวจ	แนวทางแก้/ป้องกัน
ยอดสมาชิกรวมไม่เท่ายอดบิล	หารด้วย floating point แล้วปัดยอดของแต่ละคนแยกกัน	คำนวณเป็น integer cents ต่อ item และแจก remainder ตามลำดับ deterministic เหมือน implementation ปัจจุบัน
จำนวนสินค้าไม่ปรากฏเป็น field ในประวัติ	Frontend คุณ price x quantity เป็น line total ก่อนส่ง และบันทึก quantity ไว้ใน note	นี่เป็นข้อจำกัดของ schema ปัจจุบัน หากต้อง query quantity จริงให้เพิ่ม column และ migration แยก
Confirm กดไม่ได้	ชื่อปาร์ตี้ว่าง, ไม่มี item/member หรือมี item ที่ไม่มี assignee	ตรวจ warning บนหน้า Summary และ assign ทุก item อย่างน้อยหนึ่งคน
Login เครื่องใหม่ทำให้เครื่องเดิม refresh ไม่ได้	User เก็บ refresh-token hash ได้ค่าเดียว	ยอมรับได้สำหรับ single-owner portfolio; ระบบหลายอุปกรณ์ควรมี Session table ต่อ device พร้อม revoke แยก session
หน้า History ช้าลงเมื่อข้อมูลมาก	all=true ไม่มี pagination และโหลด nested items/participants ทั้งหมด	ใช้ cursor pagination, จำกัด page size, เพิ่ม index History(userId, createdAt) และ endpoint รายละเอียด
PWA/icon ยังเป็นรูปเก่าหลัง deploy	browser/manifest cache	ตรวจ asset path, manifest และ hard refresh; เปลี่ยนชื่อไฟล์หรือ cache version เมื่อแก้ icon สำคัญ
เอกสาร HTML ใหม่แต่ PDF บน GitHub ยังเก่า	SUMMARY.html ถูก ignore ขณะที่ PDF ถูก track	สร้าง output/pdf/SUMMARY.pdf ใหม่และตรวจ render ทุกครั้งก่อน commit

16. Gap Analysis - สิ่งที่ยังขาด

ไม่มี blocker ที่ทำให้ source ปัจจุบัน build หรือ CI ไม่ผ่าน แต่ก่อนใช้เป็น production จึงควรจัดการหัวข้อต่อไปนี้

PO - ต้องทำก่อนเปิดใช้งานสาธารณะ

เรื่อง	สถานะปัจจุบัน	งานที่ควรทำ
Production credentials	ตั้งค่าจริงแล้วใน Render environment และไม่พบ secret ใน Git history	หมุน credentials หากไฟล์ JSON หลุด, เก็บสำเนาอย่างปลอดภัย และไม่อัปโหลดไฟล์จาก Downloads เข้า repository

เรื่อง	สถานะปัจจุบัน	งานที่ควรทำ
Free database expiry	Render Postgres ปัจจุบันหมดอายุ 4 ตุลาคม 2026	Export/migrate ก่อนหมดอายุ; แผนฟรีนี้ไม่เหมาะสมเป็นฐานข้อมูลถาวรและไม่มี managed backup
Persistent slip storage	เขียนลง local disk	ใช้ S3-compatible/object storage, จำกัดสิทธิ์อ่าน และ cleanup เมื่อ validation/transaction fail; ปัจจุบันไฟล์ถูกเสิร์ฟจาก <code>/uploads</code> โดยไม่มี per-file authorization
Abuse protection	ยังไม่มี rate limiting	จำกัด request ตาม IP/user โดยเฉพาะ auth และ upload
CSRF review	พึ่ง SameSite/CORS	เพิ่ม CSRF token หรือ origin enforcement หากใช้ cross-site cookies
Operations	ยังไม่มี monitoring/backup runbook	เพิ่ม structured logs, error tracking, uptime check และ restore test

P1 - เพิ่มความน่าเชื่อถือของ portfolio

เรื่อง	ช่องว่าง	งานที่ควรทำ
Real authenticated E2E	Browser tests mock unauthenticated API	เพิ่ม test account หรือ OAuth test strategy สำหรับ full create-confirm-history flow
Auth coverage	Smoke test ยังไม่ครอบคลุม refresh/logout edge cases	เพิ่ม integration tests สำหรับ rotation, revoke, expired และ reused token
Accessibility	ยังไม่มี automated audit	เพิ่ม axe และ keyboard/focus test
API contract	ยังไม่มี OpenAPI page	เพิ่ม Swagger/OpenAPI พร้อมตัวอย่าง error responses
History scaling	<code>all=true</code> โหลดทั้งหมด	เพิ่ม cursor pagination และ detail endpoint

P2 - Product evolution

- เพิ่ม payer model เพื่อระบุว่าใครสำรองจ่าย
- เพิ่ม paid/unpaid และ settlement history
- เก็บ quantity เป็น database field แทนการรวมใน price/note
- เพิ่ม tax, service charge, discount และ tip
- แชรส์บิลด้วย URL หรือ QR code
- Export ใบสรุปเป็นภาพหรือ PDF
- Sync draft ข้ามอุปกรณ์

- รองรับหลายภาษาและหลายสกุลเงินในบิลเดียว
- เพิ่มการแก้ไขหรือลบบิลที่ Confirm แล้วตาม authorization policy

Technical debt ที่ควรรู้

- อายุ cookie ใน controller ถูกกำหนดเป็น 15 นาทีและ 7 วันแยกจาก `JWT_ACCESS_TTL` และ `JWT_REFRESH_TTL` หากแก้ TTL ควรทำให้ cookie max-age ใช้ config เดียวกัน
- Refresh token hash เป็นค่าเดียวต่อ user การ login จากอุปกรณ์ใหม่อาจแทน session ของอุปกรณ์เดิม หากต้องรองรับหลายอุปกรณ์ควรมี Session model
- `confirmedAt` ถูก validate แต่ยังไม่ได้เก็บเป็น field แยก โดยระบบใช้ `createdAt` ของ History เป็นเวลาอ้างอิง
- Upload filter ตรวจสอบ MIME ที่ client ส่งและ extension แต่ยังไม่ตรวจสอบ file signature
- History API ยังไม่คืน consumption detail จึงยังใช้เปิดดู breakdown ของบิลเก่าแบบเต็มไม่ได้
- ยอดคำนวณใช้ integer cents แต่ `confirm()` ยังรวมราคาเป็น JavaScript number ก่อนสร้าง `Prisma.Decimal` ควรรวมเป็น cents เช่นเดียวกันเพื่อปิดความเสี่ยง floating-point ที่ขอบเขตฐานข้อมูล
- Google Passport flow ยังไม่ได้เปิด `state` สำหรับ OAuth login CSRF แม้ระบบจะจำกัดอีเมลเดียวและใช้ same-origin callback แล้วก็ตาม
- state-changing endpoints พัง SameSite cookie และ CORS แต่ไม่มี CSRF token/origin check โดยเฉพาะกรณีเปลี่ยนเป็น `SameSite=None`
- Party ไม่มี owner field โดยตรงและ ownership อาศัยความสัมพันธ์ผ่าน History; ถ้าเพิ่ม endpoint อ่าน/แก้ Party ตาม ID ต้องตรวจสอบ ownership ทุกครั้ง
- `partyDate` ใช้ DateTime ทั้งที่ความหมายเป็น calendar date จึงควรกำหนด timezone policy ชัดเจนก่อนรองรับผู้ใช้หลายประเทศ

17. Deployment Checklist

ก่อน deploy:

- [] CI ของ commit ที่จะ deploy ผ่านทั้งหมด
- [] ตั้ง production PostgreSQL และทดสอบ connection จาก runtime
- [] ตั้ง Google OAuth Authorized redirect URI ตรงกับ production callback
- [] ตั้ง JWT secrets แบบสุ่มอย่างน้อย 32 ตัวอักษร
- [] ตั้ง `FRONTEND_URL`, `GOOGLE_CALLBACK_URL` และ `API_PROXY_TARGET` ให้ตรงโดเมนจริง
- [] ใช้ HTTPS และตรวจสอบ cookie attributes ผ่าน browser devtools
- [] รัน `prisma migrate deploy`
- [] ย้าย slip upload ไป persistent storage หรือยืนยันว่า volume ไม่หาย
- [] ทดสอบ login, refresh, logout, create, confirm และ history ด้วยบัญชีจริง
- [] ตรวจสอบ mobile viewport และ keyboard navigation
- [] ตั้ง health monitoring และ database backup
- [] ห้ามนำ `.env` หรือ credentials เข้า Git

หลัง deploy:

- [] /health ตอบ 200
- [] /api/health ผ่าน Frontend proxy ตอบ 200
- [] OAuth callback กลับหน้า Frontend และ cookies ถูกตั้งค่าครบ
- [] Confirm บิลสำเร็จและข้อมูลปรากฏใน history
- [] Restart service แล้วประวัติและไฟล์สลิปยังอยู่
- [] ตรวจสอบ log ว่าไม่มี token, secret หรือ personal data ที่ไม่จำเป็น

18. Interview Walkthrough

ลำดับนำเสนอประมาณ 5 ถึง 7 นาที:

1. เปิด /demo อธิบายปัญหาและ UI ภายใน 30 วินาที
2. เปิด Items และ Members อธิบาย ALL กับ PARTIAL
3. เปิด calculation helper และ Backend PartyService อธิบาย integer cents กับ deterministic remainder
4. เปิด Prisma schema อธิบาย many-to-many Consumption และ transaction ตอน Confirm
5. เปิด auth flow อธิบาย same-origin proxy, HTTP-only cookies และ refresh rotation
6. เปิด GitHub Actions แสดง lint, typecheck, unit, build, integration และ E2E
7. จบด้วย Gap Analysis เพื่อแสดงว่ารู้ขอบเขตและขั้นตอน production hardening

คำตอบสั้นเมื่อถูกถามว่าอะไรคือส่วนที่ยากที่สุด:

ส่วนที่สำคัญที่สุดคือทำให้ยอดที่หารแล้วรวมกลับมาเท่ากับยอดบิลในทุกกรณี ผมจึงเปลี่ยนการคำนวณเป็น integer cents และแจกเศษตามลำดับที่กำหนด พร้อมทดสอบทั้ง unit และ API integration นอกจากนี้ยังออกแบบ callback และ API proxy ให้ cookie authentication ทำงานได้ในโครงสร้างที่ Frontend กับ Backend แยก service

19. Evidence และสถานะล่าสุด

สถานะที่ยืนยันแล้วใน commit ปัจจุบัน:

Area	ผล
Backend ESLint	ผ่าน
Backend TypeScript	ผ่าน
Backend unit tests	7 ผ่าน
Backend build	ผ่าน
Prisma migration บน CI PostgreSQL	ผ่าน

Area	ผล
API integration smoke test	ผ่าน
Frontend ESLint	ผ่าน
Frontend TypeScript	ผ่าน
Frontend unit/component tests	5 ผ่าน
Frontend production build	ผ่าน
Playwright E2E	3 ผ่าน
Archived prototype build	ผ่าน
Dependency audit ล่าสุด	ไม่พบ vulnerability ในทั้ง 3 package trees
Production Frontend	<code>https://cleft-usb2.vercel.app</code> ตอบ 200 และ deployment ล่าสุดเป็น Ready
Production Backend	<code>https://cleft-api-proc1240.onrender.com/health</code> ตอบ 200
Frontend-to-Backend proxy	<code>/api/health</code> บนโดเมน Frontend ตอบ 200
Google OAuth Client	ใช้ Client ชุดใหม่แล้ว; authorization endpoint ตอบ 200 และไม่พบ <code>deleted_client</code> หรือ <code>redirect_uri_mismatch</code>
Email access policy	production <code>ALLOWED_LOGIN_EMAIL</code> ตรงกับบัญชีเจ้าของ และ unit test ยืนยันว่า account อื่นไม่ถูก upsert
Secret hygiene	ไม่พบ Google Client Secret หรือไฟล์ <code>.env</code> จริงใน commit history; credential JSON อยู่ภายนอก repository
README	ไม่ถูกแก้ไขในรอบ portfolio

หลักฐานสำคัญใน repository:

- `.github/workflows/ci.yml`
- `backend/src/party/party.service.spec.ts`
- `backend/scripts/smoke-test.mjs`
- `frontend/test/`
- `frontend/e2e/portfolio.spec.ts`
- `frontend/app/demo/page.tsx`
- `PORTFOLIO.md`

20. Logic สำคัญที่ควรอธิบายให้ reviewer เข้าใจ

20.1 Same-origin proxy ทำให้ auth ง่ายและปลอดภัยขึ้นอย่างไร

Browser เรียก API ผ่าน `/api` บนโดเมน Vercel เดียวกับหน้าเว็บเสมอ แล้ว Next.js rewrite จึงส่งต่อไป Render Backend ข้อดีคือ client code ไม่ต้องถือ URL ภายในหลายชุด, cookie ถูกผูกกับ origin ที่ผู้ใช้เห็น และลดปัญหา third-party cookie เมื่อ Frontend/Backend อยู่คนละผู้ให้บริการ ค่า `API_PROXY_TARGET` เป็น server-side build/runtime configuration ไม่ถูก expose เป็น secret ใน browser

20.2 Authentication มีชั้นป้องกันอะไรบ้าง

1. Google ยืนยัน identity และคืน profile ที่มี email
2. Backend normalize email ด้วย trim/lowercase แล้วเทียบกับ `ALLOWED_LOGIN_EMAIL`
3. ถ้าไม่ตรงหรือ config ว่าง ระบบ throw Unauthorized ก่อน Prisma upsert จึงไม่มี user ชะงักจากบัญชีอื่น
4. เมื่อผ่านจึงสร้าง access token อายุสั้นและ refresh token อายุยาว
5. token อยู่ใน HTTP-only cookie ทำให้ JavaScript ฝั่งหน้าเว็บอ่าน token ไม่ได้โดยตรง
6. ฐานข้อมูลเก็บ SHA-256 hash ของ refresh token ไม่เก็บ token ตัวจริง
7. ทุกครั้งที่ refresh สำเร็จจะหมุน token และเขียน hash ใหม่; logout ลบ hash เพื่อ revoke

ขอบเขตที่ต้องพูดตรงไปตรงมา: hash เดียวต่อ user หมายถึงรองรับ session หลักเพียงชุดเดียว, OAuth state/CSRF protection ยังไม่สมบูรณ์สำหรับ public multi-user และ cookie max-age ยังตั้งแยกจาก TTL config

20.3 Draft, quantity และ server contract

ระหว่างกรอกบิล ข้อมูลอยู่ใน React state และ `localStorage` จึงไม่สร้าง draft records บน server และกลับมาแก้ต่อใน browser เดิมได้ ก่อน Confirm ฝั่ง Frontend แปลง `unitPrice x quantity` เป็น line total เพราะ Backend schema ปัจจุบันเก็บราคา line เดียว ไม่มี quantity column จากนั้นส่ง JSON payload กับ slip ผ่าน multipart form ข้อดีคือ contract ฝั่ง Backend เรียบง่าย แต่ข้อเสียคือ quantity กลายเป็นข้อความใน note และไม่สามารถ aggregate/report quantity จาก database อย่างเป็นโครงสร้างได้

20.4 Invariants ของการแบ่งเงิน

Invariant	วิธีรักษา	เหตุผล
ทุก item มีผู้รับผิดชอบอย่างน้อยหนึ่งคน	ตรวจทั้ง Frontend warning และ Backend <code>validateParty()</code>	ป้องกันยอดบิลส่วนที่ไม่มีใครจ่าย
ชื่อ item/member ไม่กำกวม	trim + case normalization และ reject duplicate	<code>itemNames</code> ใช้ชื่อเป็น reference ระหว่าง draft
ยอดรายคนรวมเท่ายอดบิล	แปลงแต่ละ item เป็น cents, หารด้วย floor และแจก remainder ทีละ 1 cent	หลีกเลี่ยง floating-point และการปัดแยกที่ทำให้เงินหาย/เกิน
ผลการแจกเศษทำซ้ำได้	แจกตามลำดับ assignee ที่แน่นอน	same input ต้องได้ same output และเขียน test ได้

Invariant	วิธีรักษา	เหตุผล
ข้อมูล Confirm ไม่บันทึก ครั้งสุดท้าย	สร้าง Party, Items, Participants, Consumptions และ History ใน Prisma transaction	ถ้า query ใดล้มเหลว database rollback ทั้งหมด

20.5 Data ownership และ History

History query filter ด้วย `userId` จาก JWT ไม่รับ user ID จาก client จึงไม่สามารถขอประวัติของคนอื่นโดยเปลี่ยน query parameter ได้ ความเป็นเจ้าของ Party ปัจจุบันเชื่อมผ่าน History ดังนั้น endpoint ใหม่ที่รับ `partyId` ต้อง join/ตรวจ History ของ user ก่อนอ่าน แก้ หรือลบ ห้าม query Party ตาม ID แล้วคืนตรง ๆ

20.6 Fail-fast configuration และ delivery

`validateEnv()` ทำให้โปรแกรมไม่ยอม start หากขาด database, OAuth, allowlist หรือ JWT secrets และเพิ่มเกณฑ์ secret ขั้นต่ำใน production วิธีนี้เปลี่ยน configuration error ให้เห็นทันทีแทนการรอฟังตอนผู้ใช้งาน flow สำคัญ Docker image รัน `prisma migrate deploy` ก่อน start ขณะที่ GitHub Actions ตรวจ lint, typecheck, unit, build, migration, API integration และ browser flow แยกเป็น jobs

21. Scalability Assessment

คำตอบสั้น: โครงสร้างโค้ด scale ต่อได้ แต่ deployment ปัจจุบันตั้งใจสำหรับ portfolio/private demo เท่านั้น และ effective authenticated-user capacity เท่ากับ 1 บัญชีเพราะ email allowlist ก่อนจะพูดถึงขีดจำกัดเครื่องเสียอีก

21.1 ระดับที่รองรับได้ในสภาพปัจจุบัน

ระดับ	ความเหมาะสม	ประมาณการแบบมีเงื่อนไข	ข้อจำกัดหลัก
Portfolio / reviewer demo	เหมาะ	เจ้าของ 1 คนและ reviewer เปิด <code>/demo</code> ; การใช้งานเป็นครั้งคราวสบาย	request แรกอาจรอ cold start ประมาณหนึ่งนาที
กลุ่มทดสอบเล็ก	พอทดลองได้หลังเปลี่ยน allowlist	ประมาณ 1-10 concurrent active users หรือหลักร้อย sessions/วันเมื่อกระจายเวลา เป็นเพียง engineering estimate ไม่ใช่ผล load test	0.1 CPU/512 MB backend, database 256 MB, ไม่มี pooling และไม่มี pagination
แอปกลุ่มเล็กใช้งานจริง	ต้องย้ายออกจาก Free Postgres และแก้ PO	หลักสิบ concurrent users และหลักพัน request/วันมีโอกาสรองรับได้บน instance เริ่มต้นแบบ paid แต่ต้อง load test	object storage, indexes, rate limit, monitoring, backup และ multi-session

ระดับ	ความเหมาะสม	ประมาณการแบบมีเงื่อนไข	ข้อจำกัดหลัก
สาธารณะระดับหลายร้อย concurrent	สถาปัตยกรรมต่อยอดได้ แต่ยังไม่พร้อม	ต้องวัด p95 latency/error rate หลัง scale instance และ connection pool	horizontal scaling, DB capacity/indexes, stateless upload, queue และ observability
หลักพัน concurrent ขึ้นไป	ต้องออกแบบ production platform เพิ่ม	ไม่มีข้อมูล benchmark ให้รับรอง	cache, read model, autoscaling, load shedding, CDN strategy, job queue, HA database และ incident process

ตัวเลขข้างต้นเป็นช่วงประเมินเพื่อสื่อสาร ไม่ใช่ SLA เพราะ repository ยังไม่มี load/stress test สิ่งที่ต้องดีกว่าการอ้างจำนวน user แบบตายตัวคือกำหนด workload แล้ววัด requests/second, p50/p95/p99 latency, error rate, CPU, memory, database connections และ query time

21.2 ลำดับ bottleneck ที่จะเจอก่อน

1. **Access policy:** `ALLOWED_LOGIN_EMAIL` จำกัดไว้หนึ่งบัญชี จึงต้องเปลี่ยนเป็น invitation/allowlist table หรือเปิดรับทุก Google account ตาม product policy
2. **Database lifetime:** Free Postgres หมดอายุ 4 ตุลาคม 2026 ทำให้ scale ระยะเวลาไม่ได้ แม้ traffic ต่ำ
3. **Cold start:** Backend spin down หลัง idle 15 นาทีและ warm-up อาจประมาณหนึ่งนาที
4. **Compute:** Render Free มี 0.1 CPU, RAM 512 MB และ scale เกินหนึ่ง instance ไม่ได้
5. **Storage:** upload อยู่ local ephemeral filesystem จึงขัดกับทั้ง persistence และ horizontal scaling
6. **Query:** `history?all=true` โหลดทุก record พร้อม nested relations และยังไม่มี composite index สำหรับ user/time
7. **Session:** refresh hash เดียวต่อ user ทำให้ multi-device และ session administration จำกัด

21.3 Roadmap เพื่อ scale โดยไม่ rewrite ทั้งระบบ

1. ย้าย slip ไป object storage แบบ private และใช้ signed URL
2. ย้าย PostgreSQL ไปแผนถาวร เปิด backup/PITR ตามงบ และเพิ่ม `History(userId, createdAt)` index
3. เพิ่ม cursor pagination, limit สูงสุด และ endpoint รายละเอียดบิล
4. เพิ่ม Session model ต่ออุปกรณ์, OAuth state, CSRF/origin protection และ rate limiting
5. เปลี่ยน email gate เป็น policy ที่ตั้งใจ เช่น invitation table, admin role หรือ domain allowlist
6. เพิ่ม structured logging, error tracking, uptime monitor และ dashboard ของ latency/database
7. ทำ k6/Artillery load test บน staging แล้วเลือก vertical/horizontal scale จากข้อมูลจริง

ส่วนที่ช่วยให้ต่อขยายได้ง่ายคือ Backend ไม่มี in-memory business state, draft อยู่ client และข้อมูลหลักอยู่ PostgreSQL เมื่อย้าย upload ออกจาก local disk แล้วแต่ละ backend instance จะเกือบ stateless จึง scale horizontally ได้โดยไม่เปลี่ยน product flow หลัก

22. ค่าใช้จ่ายและเงื่อนไขแผนฟรี ณ 5 กันยายน 2026

22.1 สถานะ resource ที่ตรวจจากบัญชีจริง

บริการ	แผนปัจจุบัน	ค่าใช้จ่ายฐาน	ข้อจำกัด/ความเสี่ยง
Vercel Frontend	Hobby	0 USD/เดือน	ใช้สำหรับ personal/non-commercial; เมื่อเกิน included usage โดยทั่วไป feature/project จะถูก pause แทนการคิด overage บน Hobby
Render Backend	Free, 1 instance, Singapore	0 USD/เดือน	0.1 CPU, 512 MB, spin down หลัง idle 15 นาที, ไม่มี persistent disk และ scale เกินหนึ่ง instanceไม่ได้
Render PostgreSQL	Free, PostgreSQL 18, Singapore	0 USD จนหมดอายุ	0.1 CPU, 256 MB, 1 GB, สูงสุด 100 connections, ไม่มี managed backup/pooling และหมดอายุ 4 ตุลาคม 2026
Google OAuth	Identity scopes: email/profile	โค้ดนี้ไม่ได้เรียก paid Google API	ถ้า Audience เป็น Testing จำกัด 100 test users และ authorization ของ test user หมดอายุใน 7 วัน; ปัจจุบัน backend รับเพียงบัญชีเดียว
GitHub Actions	Public repository + standard Ubuntu runners	ฟรี	larger runners หรือบริการเสริมแบบคิดเงินไม่รวมอยู่ในการตั้งค่าปัจจุบัน
Custom domain	ยังใช้ subdomain ฟรี	0 USD	ถ้าซื้อโดเมนภายหลังมีค่าจด/ต่ออายุแยกจาก hosting

22.2 คำตอบว่า “แพลนนี้จะเสียเงินไหม”

จาก resource ที่เลือกอยู่ตอนนี้ ค่าใช้จ่ายฐานคือ 0 USD/เดือน และเหมาะกับการเปิด portfolio แบบ traffic ต่ำ แต่ไม่ควรแปลว่าไม่มีทางเกิดค่าใช้จ่ายทุกกรณี:

- Vercel Hobby ไม่คิด on-demand overage แบบ Pro; เมื่อเกินโควตาจะถูกจำกัดหรือ pause จนรอบ usage reset จึงควรคง plan เป็น Hobby และไม่เริ่ม Pro trial โดยไม่ตั้งใจ
- Render ระบุว่า Free service อาจคิดค่า outbound bandwidth/build pipeline ส่วนเกินหาก workspace มี payment method; ถ้าไม่มี payment method ระบบจะ suspend/หยุด build แทน ควรตรวจ Billing, payment method, usage notification และ spend limit ใน Dashboard
- ฐานข้อมูลจะไม่แอบเปลี่ยนเป็น paid เองตามข้อมูลที่เห็น แต่จะหมดอายุและถูกลบหลัง grace period หากไม่ upgrade ความเสี่ยงหลักจึงเป็นข้อมูลหาย ไม่ใช่บิล
- อย่าเปิด uptime ping ถี่ ๆ เพื่อกัน sleep โดยไม่จำเป็น เพราะ service ที่รันอยู่กิน 750 free instance hours ที่แชร์ทั้ง workspace
- อย่าเพิ่ม paid database, persistent disk, autoscaling, Pro team, paid marketplace storage หรือ custom domain โดยไม่ตรวจหน้าสรุปราคาก่อนกดยืนยัน

22.3 โควตาที่เกี่ยวกับโปรเจกต์นี้

- Vercel Hobby: 100 GB Fast Data Transfer, สูงสุด 1,000,000 Edge Requests, 6,000 build execution minutes และ 100 deployments/วันตามเอกสารปัจจุบัน
- Render Free: 750 instance hours ต่อ workspace ต่อเดือน; ชั่วโมงรีเซตทุกต้นเดือนและไม่ทบ
- Render Free Postgres: ใช้ได้หนึ่งฐานต่อ workspace, 1 GB และมีอายุ 30 วัน
- Google OAuth Testing: สูงสุด 100 test users; identity-only flow นี้ไม่ใช่ bottleneck ก่อน backend allowlist

แหล่งข้อมูลทางการ: [Vercel Hobby](#), [Vercel Limits](#), [Render Free](#), [Render Compute Plans](#), [Google Auth Audience](#) และ [GitHub Actions Billing](#)

23. Final Portfolio Readiness Checklist

พร้อมแล้ว

- [x] Frontend, Backend และ proxy health ทำงานบน production
- [x] Google OAuth Client ใหม่ไม่ขึ้น `deleted_client` หรือ redirect mismatch
- [x] จำกัด login เฉพาะบัญชีเจ้าของและมี unit test ป้องกัน regression
- [x] CI ครอบคลุม lint, typecheck, unit, build, database migration, integration smoke และ Playwright
- [x] dependency ทั้ง 3 package trees ไม่มี known vulnerability จาก npm audit หลังอัปเดต
- [x] ไม่พบ Google Client Secret หรือไฟล์ environment จริงใน Git history
- [x] มีหน้า `/demo` ให้ reviewer ดู UI โดยไม่ต้องใช้บัญชีส่วนตัว
- [x] มี PDF อธิบาย architecture, logic, trade-offs, scaling และ operational risks

ควรทำก่อนส่งลิงก์

- [] เปิด production จาก incognito และ mobile จริงหนึ่งรอบ: Home, Demo, Google login, Items, Members, Summary, Confirm และ History
- [] กด logout แล้วตรวจว่า protected pages กลับหน้า login
- [] ลบหรือเก็บไฟล์ OAuth client JSON ใน Downloads ไว้ใน password manager/secure storage ห้ามแนบใน GitHub หรือ portfolio
- [] ตรวจ Google Audience ว่า account เจ้าของเป็น Test user หาก app ยังเป็น Testing
- [] ตรวจ Render Billing ว่าไม่มี payment method หรือกำหนด guardrails ตามที่ต้องการ
- [] export ฐานข้อมูลหรือวางแผนย้ายก่อน 4 ตุลาคม 2026

ไม่บล็อกการยื่นฝึกงาน แต่ต้องบอกตามจริง

- slip upload ไม่ถาวรและ URL ไม่มี per-file authorization
- ไม่มี real Google-authenticated E2E automation, accessibility audit, load test และ monitoring
- ยังไม่มี pagination, rate limiting, OAuth state/CSRF token และ multi-device session model
- ระบบเป็น bill calculator/history ไม่ใช่ payment gateway หรือ debt settlement platform

วิธีนำเสนอที่ดีคือแยกคำว่า **portfolio-ready** ออกจาก **public-production-ready** ให้ชัด แสดงสิ่งที่ทำงานจริงพร้อมหลักฐาน และอธิบาย backlog ตามความเสี่ยง/ผลกระทบ แทนการอ้างว่าระบบสมบูรณ์ทุกด้าน

24. Final Assessment

ในมุมมอง portfolio โปรเจกต์นี้อยู่ในระดับที่ยื่นสมัครฝึกงานได้แล้ว เพราะมี product flow ที่เข้าใจง่าย สถาปัตยกรรม Full Stack ที่ deploy จริง business logic การเงินที่อธิบายและทดสอบได้ การจัดการ relational data/transaction, authentication, responsive UI, CI และหน้า demo สำหรับ reviewer

ระดับการ scale ปัจจุบันคือ private portfolio demo และกลุ่มทดสอบขนาดเล็ก ไม่ใช่ public production: authenticated access จำกัดหนึ่งบัญชี, Render Free มี cold start/เครื่องขนาดเล็ก, Postgres หมดอายุ, upload ไม่ถาวร และยังไม่มี operational controls บางส่วน ข้อดีคือ service boundary และ data model ไม่ได้ตันเชิงสถาปัตยกรรม สามารถเพิ่ม object storage, persistent database, pagination/indexes, session model และ horizontal scaling โดยไม่ต้องเขียน product ใหม่ทั้งหมด

ค่าใช้จ่ายฐานปัจจุบันเป็น 0 USD/เดือนตราบิตที่ยังคง Vercel Hobby และ Render Free แต่ต้องเฝ้า Render overage/payment method และวันหมดอายุฐานข้อมูล การระบุเงื่อนไขเหล่านี้โดยตรงไปตรงมาแสดงให้ผู้สัมภาษณ์เห็นทั้งทักษะการ implement และวิธีคิดด้าน reliability, security, scalability และ cost ownership